

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

- Fechas de entrega y penalización:

Hasta el martes 26 de noviembre, 16:00 horas	0 %
Hasta el miércoles 27 de noviembre, 16:00 horas	20 %
Hasta el jueves 28 de noviembre, 16:00 horas	40 %

Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de
`http://deliverit.fi.upm.es`
- El/los fichero(s) que hay que subir es/son
`TreePriorityQueue.java`.
- La clase debe estar en el paquete `aed.treepriorityqueue` .
- La documentación de la API de `aedlib.jar` está disponible en
`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Colas con Prioridad

- En una cola FIFO el primer elemento en entrar es el primero en salir
- En las *colas con prioridad* el orden de salida viene determinado por la *prioridad* del elemento
- Se puede ver una cola con prioridad como una estructura de datos en la que los elementos se almacenan en orden de prioridad
 - ▶ A nivel de implementación no es necesario que esto ocurra así, lo importante es que la cola devuelva primero el elemento con más prioridad (ojo!)
- Las *entradas* de una cola con prioridad tienen
 - ▶ Una *clave* que indica la prioridad del elemento
 - ▶ Un *valor* que indica el elemento a insertar
 - ▶ Al par *clave-valor* lo llamamos *entrada* (entry)

Implementar una cola de prioridad usando un montículo

- La tarea de hoy es implementar una API reducida de un cola de prioridad:

```
public interface PriorityQueue<K,V> implements Iterable<Entry<K,V>> {  
    // Devuelve del numero de elementos contenidos en la cola  
    public int size();  
    // Devuelve true si y solo si esta vacia  
    public boolean isEmpty();  
  
    // Devuelve el elemento con la clave con mayor prioridad (menor clave)  
    public Entry<K,V> first() throws EmptyPriorityQueueException;  
    // Añade el elemento v con prioridad k  
    public void enqueue(K k, V v);  
    // Devuelve y elimina el elemento con mayor prioridad (menor clave)  
    public Entry<K,V> dequeue() throws EmptyPriorityQueueException;  
}
```

Implementar una cola de prioridad usando un montículo

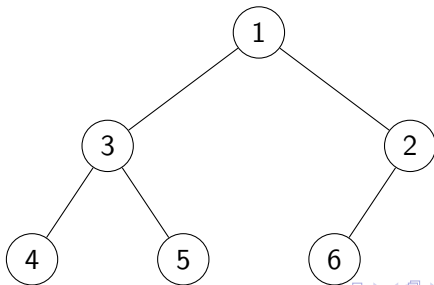
- Implementaremos la cola de prioridad usando un montículo
- La clase `TreePriorityQueue` exige que las claves sean `Comparable`

```
public class TreePriorityQueue<K extends Comparable<K>,V>...
```

para que las podamos ordenar en el montículo

¿Que es un montículo?

- Un árbol binario con restricciones en su estructura:
 - ▶ Todos los niveles del árbol están llenos, excepto quizás el último.
 - ▶ Si hay “huecos libres”, están en la parte derecha del último nivel.
- Los nodos guardan entradas $\text{Entry}\langle K, V \rangle$. Para crear una entrada usad la clase `EntryImpl`.
- Restricción en el contenido: la clave de la entrada de cada nodo es mayor o igual que la clave de la entrada de su padre (si lo tiene – *heap-order property*).
- Un ejemplo (mostrado solo las claves):



Implementación de un montículo

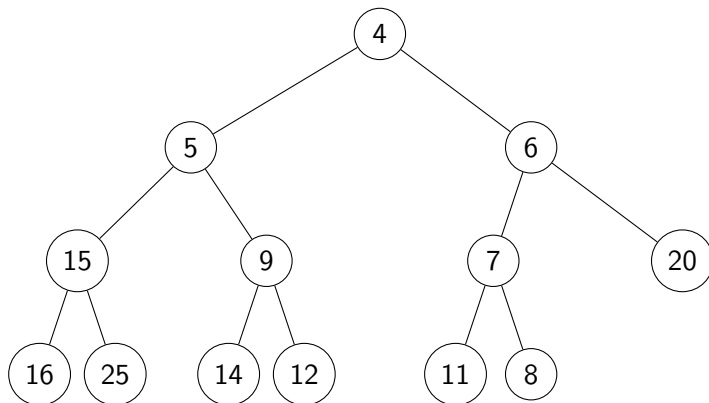
- La implementación más habitual de un montículo usa un array.
- En este laboratorio vamos implementar un montículo mediante un árbol binario.

Operación enqueue()

- Insertar un dato crea un nuevo nodo en la posición libre más a la izquierda del último nivel (la única que no viola la propiedad estructural del montículo).
- Como esto puede *violar* la *heap-order property* puede ser necesario reajustar los nodos del árbol
 - ▶ Se comprueba la “heap-order property” entre el nuevo nodo y su padre
 - ★ Si se cumple hemos acabado
 - ★ Si no se cumple entonces se intercambian las entradas entre el nodo nuevo y el padre
 - ▶ Se repite la operación con el nodo intercambiado y el padre correspondiente hasta que se cumpla la “heap-order property” o hasta llegar a la raíz
- Esto se conoce como *up-heap bubbling*

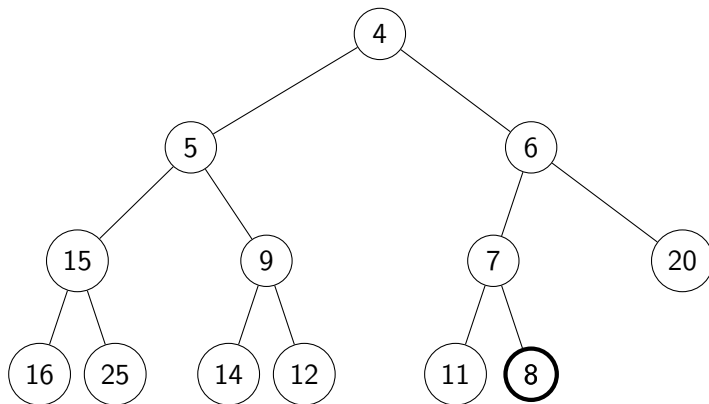
up-heap bubbling

Insertamos la clave 2



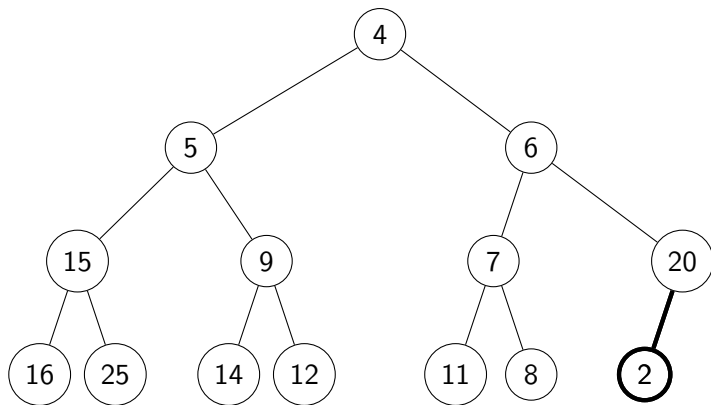
up-heap bubbling

Insertamos la clave 2



up-heap bubbling

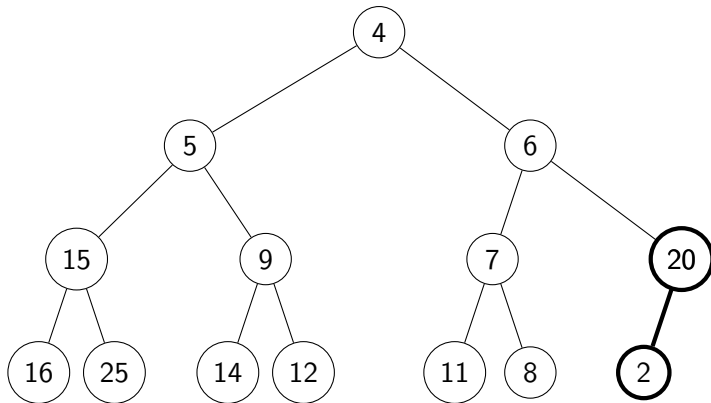
Insertamos la clave 2



Insertamos el 2 en un nuevo nodo en la *última* posición libre

up-heap bubbling

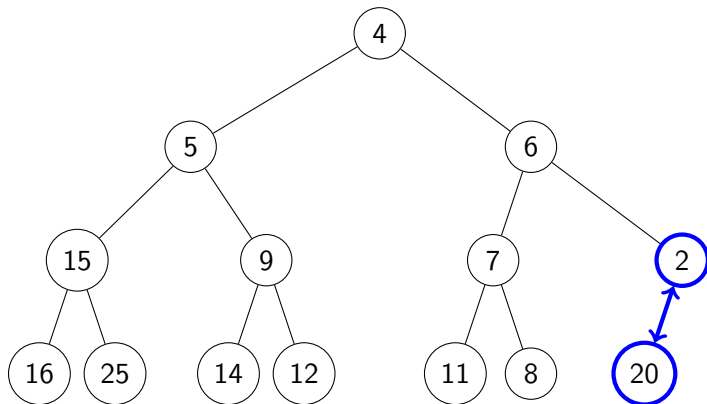
Insertamos la clave 2



¿Violamos la heap-order property?

up-heap bubbling

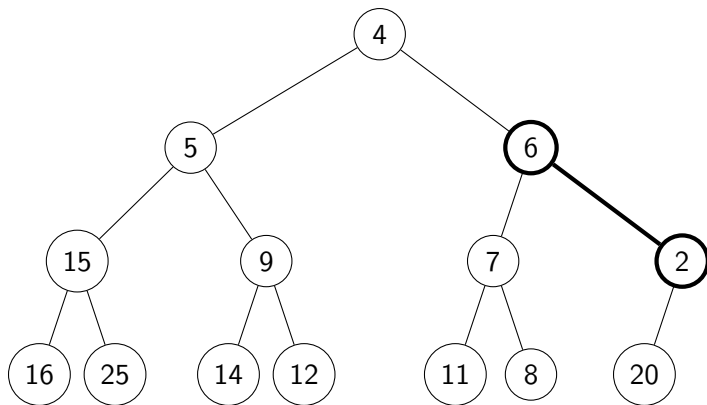
Insertamos la clave 2



Intercambiamos los nodos

up-heap bubbling

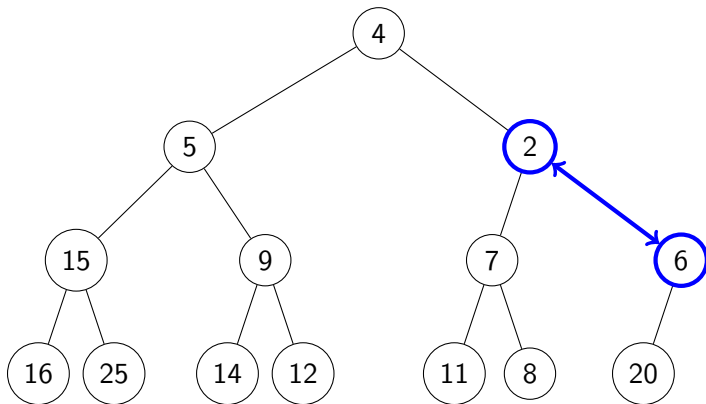
Insertamos la clave 2



¿Violamos la heap-order property?

up-heap bubbling

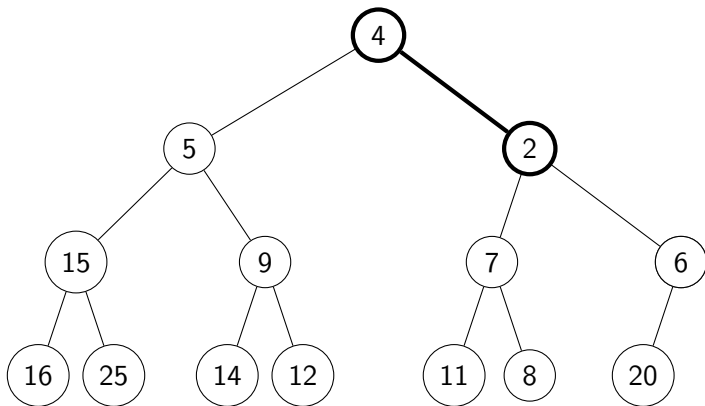
Insertamos la clave 2



Intercambiamos los nodos

up-heap bubbling

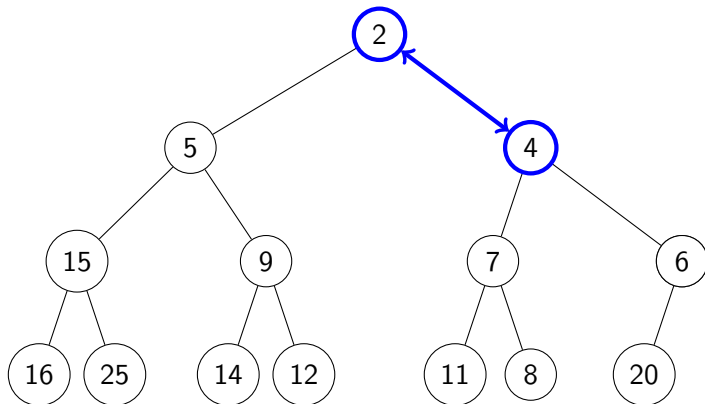
Insertamos la clave 2



¿Violamos la heap-order property?

up-heap bubbling

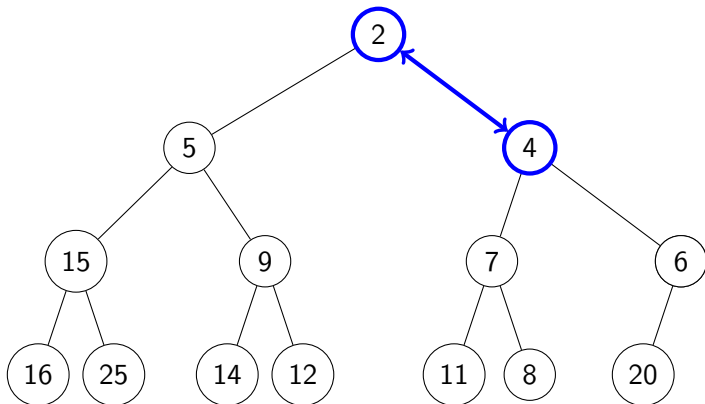
Insertamos la clave 2



Intercambiamos los nodos

up-heap bubbling

Insertamos la clave 2



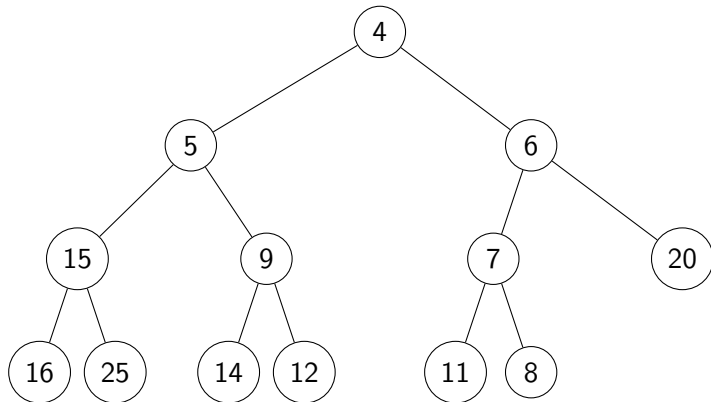
FIN

(no siempre hay que subir hasta la raíz)

Operación dequeue()

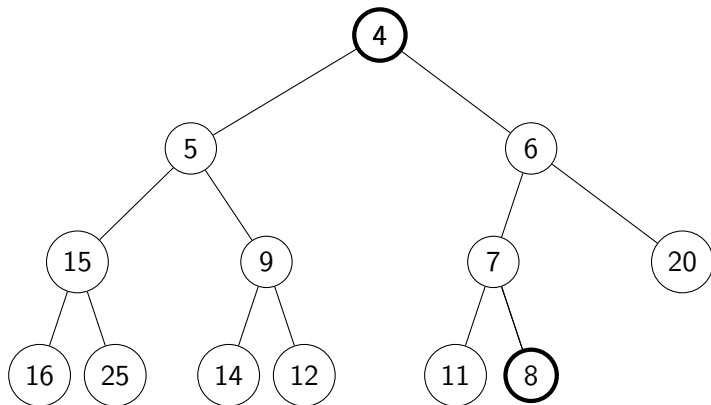
- La entrada de menor clave es la raíz, pero no se puede borrar directamente.
- *Llevamos* el contenido del *último nodo* a la *raíz* y borramos ese último nodo.
- Se puede *violar* la *heap-order property* y tener que reajustar los nodos del árbol.
 - ▶ Comprobamos la *heap-order property* entre la raíz y sus hijos.
 - ★ Si se cumple, hemos acabado.
 - ★ Si no se cumple, se intercambiamos las entradas entre la raíz y el nodo hijo de **menor** clave.
 - ▶ Se repite la operación con el hijo intercambiado y sus hijos hasta que se cumpla la “heap-order property” o hasta llegar a una hoja.
- Esto se conoce como *down-heap bubbling*

Extraer el elemento menor: *down-heap bubbling*



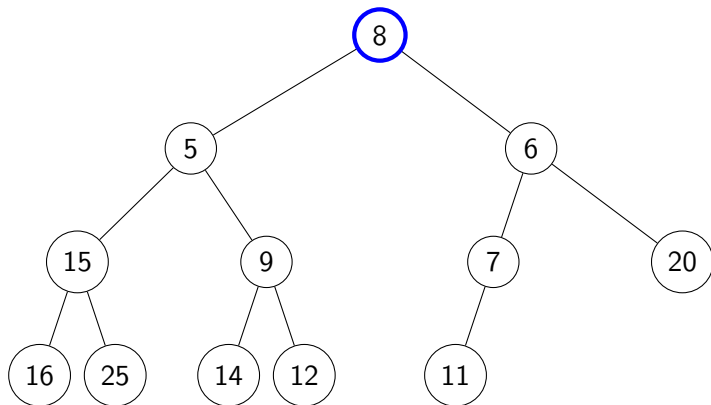
Ejecutamos *dequeue()*

Extraer el elemento menor: *down-heap bubbling*



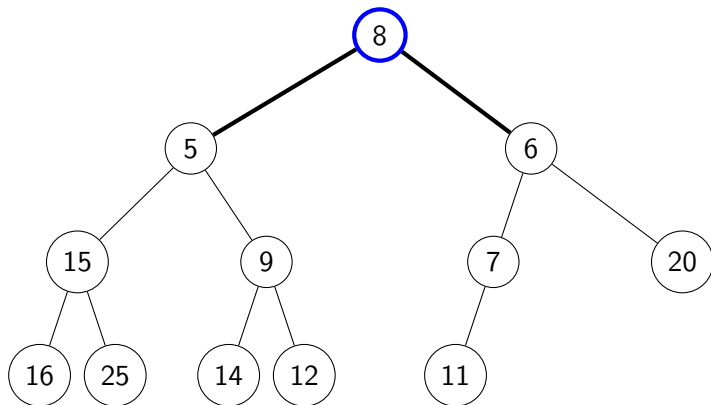
Quitamos la raíz y ponemos el último como raíz

Extraer el elemento menor: *down-heap bubbling*



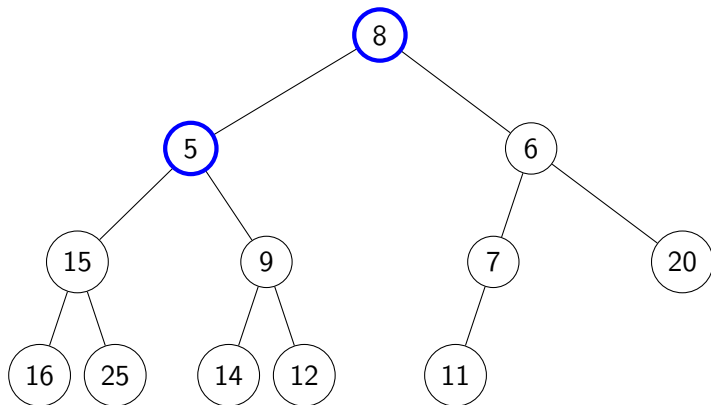
Quitamos la raíz y ponemos el último como raíz

Extraer el elemento menor: *down-heap bubbling*



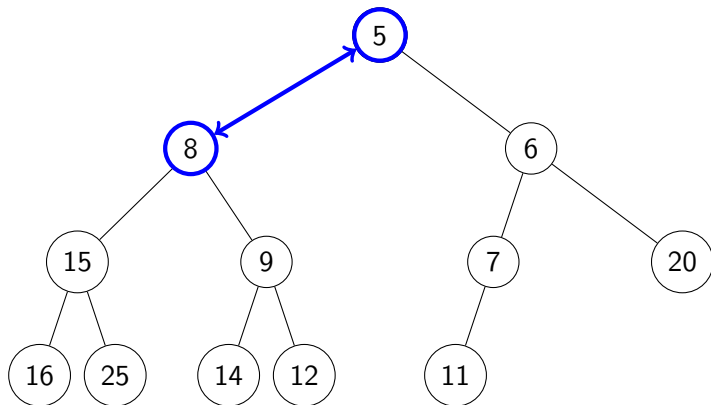
¿Violamos la heap-order property?

Extraer el elemento menor: *down-heap bubbling*



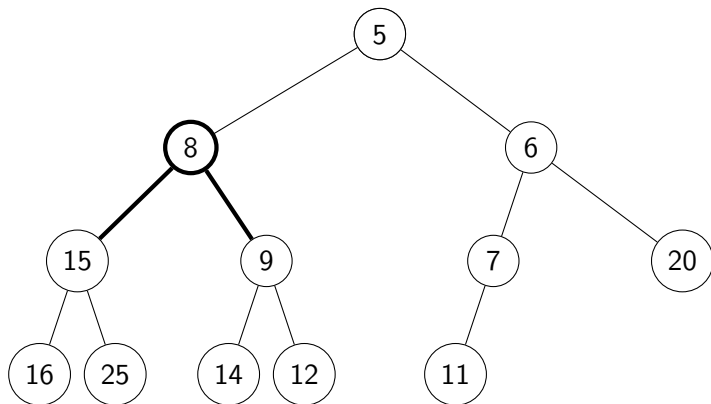
Intercambiamos con el hijo con mayor prioridad

Extraer el elemento menor: *down-heap bubbling*



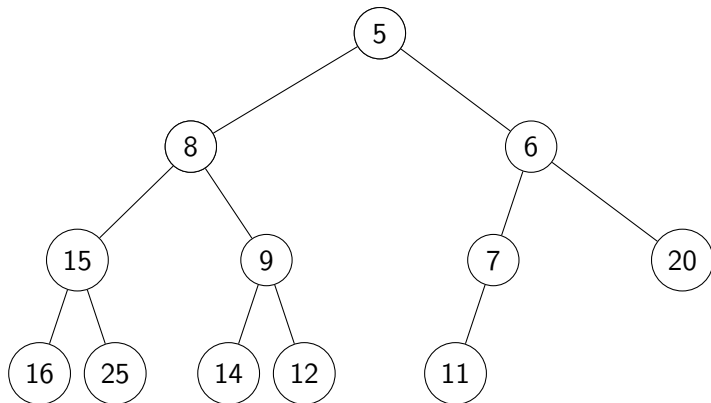
Intercambiamos con el hijo con mayor prioridad

Extraer el elemento menor: *down-heap bubbling*



¿Violamos la heap-order property?

Extraer el elemento menor: *down-heap bubbling*



FIN

Nodos importantes

- ¿Donde esta el “último” nodo del árbol? (importante para `dequeue()`)
- ¿Donde deberíamos insertar un nodo nuevo? (importante para `enqueue()`)
- La clase `TreePriorityQueue` contiene un atributo

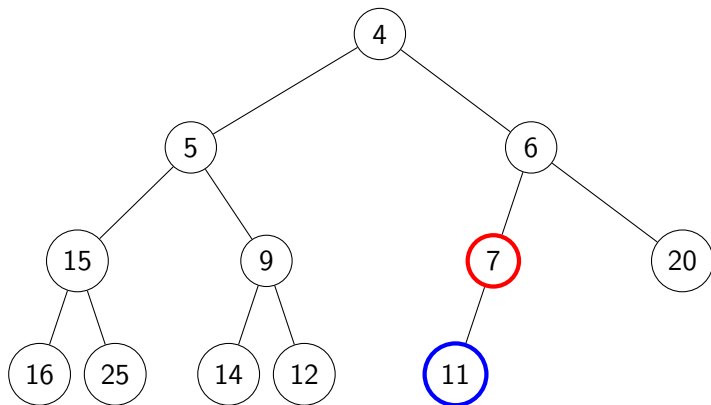
`Position<Entry<K,V>> lastPos;`

que debe apuntar **siempre** a la última posición del árbol.

- A continuación veremos como se puede:
 - ▶ Dada la última posición, encontrar dónde insertar un nodo nuevo.
 - ▶ Tras borrar un elemento, encontrar la nueva última posición.

¿Dónde insertar?

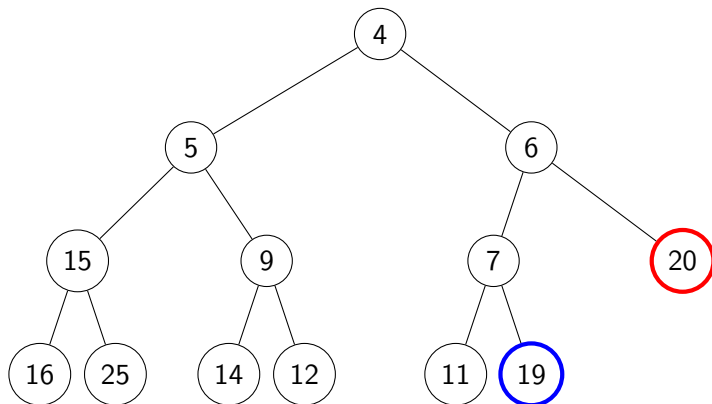
- ¿Cuál es el nodo padre del siguiente nuevo nodo, dada la última posición del árbol?



11: última posición; 7: dónde insertar el siguiente nodo

¿Dónde insertar?

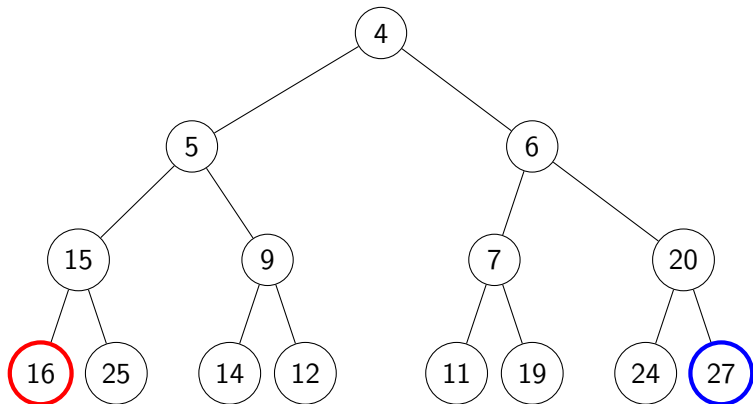
- ¿Cuál es el nodo padre del siguiente nuevo nodo, dada la última posición del árbol?



19: última posición; 20: dónde insertar el siguiente nodo

¿Dónde insertar?

- ¿Cuál es el nodo padre del siguiente nuevo nodo, dada la última posición del árbol?



27: última posición; 16: dónde insertar el siguiente nodo

¿Dónde insertar?

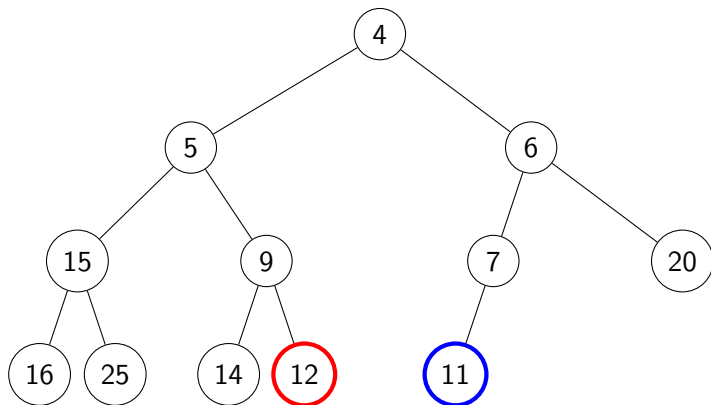
- ¿En qué nodo n debemos insertar el nuevo nodo? Intuición:
 - ▶ Si no hay nodos a la derecha del último nodo, n es el nodo más a la izquierda del árbol.
 - ▶ Si no, n es el nodo más a la izquierda que no tiene dos hijos y que está a la derecha del último nodo.

Idea de un algoritmo:

- Si solo tenemos la raíz, se trata separadamente.
- Vamos subiendo desde el último nodo hacia la raíz. Cada vez que llegamos al padre p de un nodo:
 - ▶ Si llegamos a p desde su hijo derecho:
 - ★ Si p no es la raíz, seguimos subiendo.
 - ★ Si p es la raíz, n es el nodo de más a la izquierda del árbol.
 - ▶ Si llegamos a p desde su hijo izquierdo:
 - ★ Si p no tiene hijo derecho, $n = p$.
 - ★ Si no, n es el nodo de más a la izquierda de hijo derecho de p .

¿Dónde va a estar el nuevo último nodo?

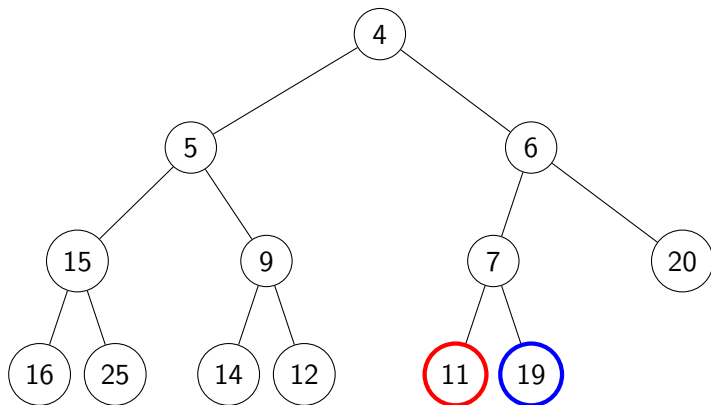
- Tras borrar el último nodo, ¿cuál es ahora el último nodo?



11: último nodo antes de borrar; 12: último nodo tras borrar 11

¿Donde va a estar el nuevo último nodo?

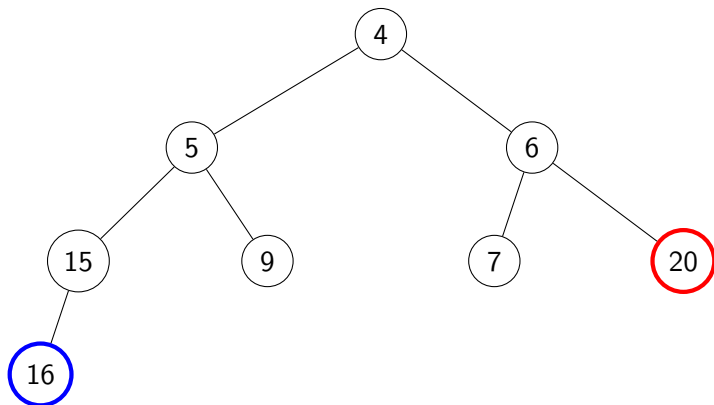
- Tras borrar el último nodo, ¿cuál es ahora el último nodo?



19: último nodo antes de borrar; 11: último nodo tras borrar 19

¿Dónde va a estar el nuevo último nodo?

- Tras borrar el último nodo, ¿cuál es ahora el último nodo?



16: último nodo antes de borrar; 20: último nodo tras borrar 16

¿Cuál va a ser el nuevo último nodo n cuando borramos el anterior?

- Intuición para determinar n :

- ▶ Es el nodo hoja más a la derecha que está a la izquierda del último nodo.
- ▶ Pero si no hay ningún nodo a la izquierda del último nodo, se elige el nodo hoja mas a la derecha del árbol.

Idea de un algoritmo:

- Si solo tenemos la raíz, la tratamos de forma especial.
- Subimos desde el último nodo hacia la raíz. Cada vez que llegamos al padre p de un nodo:
 - ▶ Si llegamos a p desde su hijo izquierdo:
 - ★ Si p no es la raíz, seguimos subiendo.
 - ★ Si p es la raíz, n es el nodo más a la derecha del árbol.
 - ▶ Si llegamos a p desde su hijo derecho:
 - ★ n es el nodo más a la derecha del subárbol izquierdo de p .

Notas Generales

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test `TesterLab5` correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final

¡Seguir estos consejos os permitirá conseguir mejores resultados!

- Corrección
- Ausencia de código repetido con la misma funcionalidad (podéis usar métodos auxiliares para evitarlo)
- Concisión y claridad del código
- Legibilidad, incluida selección de nombres descriptivos para variables y métodos
- El código debe estar correctamente indentado y con comentarios *útiles* cuando lo veáis necesario
- Eficiencia:
 - ▶ Se valorará la complejidad computacional del código
 - ▶ Se valorará no iterar innecesariamente en los recorridos de las estructuras de datos